CodeGraph Incorporation — Design Spec

- **Date:** 2026-05-20
- **Status:** Approved (operator sign-off, brainstorming session "Lava")
- **Author:** Claude Code (Opus 4.7), operator-directed
- **Spec type:** project-specific tooling incorporation
- **Classification:** project-specific (per HelixConstitution §11.4.17 — codegraph is a Lava developer-tooling choice; the anti-bluff verification *pattern* it follows is universal)

1. Context

The operator directed incorporation of **codegraph** (`https://github.com/colbymchenry/codegraph`, npm package `@colbymchenry/codegraph`) into the Lava project. CodeGraph is a Node.js 18+ tool that builds a **local SQLite semantic knowledge graph** of a codebase and exposes it to AI coding agents over the **Model Context Protocol (MCP)**. It is 100% local — no cloud, no external API — which aligns with Lava's Local-Only CI/CD constitutional constraint.

The operator's primary CLI agent is **Claude Code**; codegraph must additionally work with **OpenCode, Kimi CLI, Crush, and Qwen Code**. All five are installed on the host and were probed: all five support non-interactive invocation and MCP server registration.

This incorporation is performed under the standing §6.L Anti-Bluff Functional Reality Mandate (this is the **63rd invocation**). The verification suite is the load-bearing deliverable: a green test must mean codegraph genuinely works for a real agent end-to-end, not that "the wiring compiles."

2. Goal

1. Install codegraph globally on the host (`npm install -g @colbymchenry/codegraph`).
2. Initialize + index the Lava domain codebase into a real, non-empty knowledge graph.

3. Wire codegraph's MCP server into all five CLI agents, project-scoped + committed where each agent supports it.
4. Build an **anti-bluff verification suite** that proves, with falsifiability rehearsals, that codegraph works end-to-end for every agent.
5. Document everything (`docs/CODEGRAPH.md`).
6. Propagate the §6.L mandate (63rd invocation) and create `QWEN.md` across root + all submodules + `lava-api-go`.
7. Commit and push parent + every touched submodule to GitHub + GitLab; verify §6.C convergence.

3. Non-goals

- codegraph does **not** become a `vasic-digital` submodule. It is third-party developer tooling (like `gh`, `glab`, Gradle) — the Decoupled Reusable Architecture rule does not require submodule extraction for external tools.
- No new Git remote on any provider (§6.W preserved).
- Indexing the 18 pinned submodules is out of scope for the initial index — they are frozen external code with their own constitutions. The index covers Lava domain code only.
- This is not a distributable artifact — §6.P (versionCode/changelog) does not apply; §6.S (CONTINUATION.md) does.

4. Constitutional compliance

Rule	How satisfied
§6.W (GitHub+GitLab only)	codegraph is an npm tool; no Git remote added.
§6.U (no sudo/su)	npm global prefix is user-writable; install needs no elevation.
§6.R (no hardcoding)	`.codegraph/config.json` uses project-relative paths; MCP configs reference `codegraph` on `PATH`; no IPv4/host:port/UUID literals.
§6.H (credential inviolability)	`.codegraph/config.json` excludes `.env*`, `keystores/`, `app/google-services.json`, `*.keystore` from indexing.
Local-Only CI/CD	codegraph is 100% local; the verification suite runs via `scripts/`.
Decoupled Reusable Architecture	codegraph is external tooling, not Lava domain code, not reusable `vasic-digital` code → no submodule.
§6.J / §6.L (anti-bluff)	Verification suite carries falsifiability rehearsals; primary assertions on real

Rule	How satisfied
	codegraph-derived `file:line` data; per-agent end-to-end drives.
§6.AD (HelixConstitution inheritance)	`QWEN.md` joins the inherited per-scope doc set; no inherited rule weakened.

5. Architecture

```
.codegraph/
  config.json        # committed — language/exclude/framework
config
  codegraph.db       # gitignored — SQLite knowledge graph (build
artifact)

Agent MCP wiring (project-scoped, committed where supported):
  .mcp.json          # Claude Code  — { mcpServers:
{ codegraph: ... } }
  opencode.json      # OpenCode     — { mcp: { codegraph: ... } }
  .qwen/settings.json# Qwen Code    — { mcpServers:
{ codegraph: ... } }
  .crush.json        # Crush        — { mcp: { codegraph: ... } }
  <kimi project cfg> # Kimi CLI     — MCP server entry

Each agent spawns `codegraph serve --mcp` (stdio transport) and
gets the
codegraph_search / codegraph_context / codegraph_callers /
codegraph_callees /
codegraph_impact / codegraph_node / codegraph_files /
codegraph_status tools.

Verification:
  scripts/verify-codegraph.sh        # runner — all 5 layers
  tests/codegraph/*.sh               # per-layer test scripts
  .lava-ci-evidence/codegraph/       # evidence output
```

6. Phases

Phase 0 — Precondition: committed + pushed

Main repo verified 0-ahead/0-behind on GitHub and GitLab. Verify each of the 18 submodules' pinned commit exists on its own GitHub + GitLab remotes (§6.C convergence); push any divergence. Untracked build artifacts (`submodules/challenges/Panoptic/`, `submodules/containers/cmd/emulator-matrix/emulator-matrix`) are **not** committed.

Phase 1 — Install codegraph

`npm install -g @colbymchenry/codegraph`. Verify `codegraph --version`. Honest risk: native modules (`better-sqlite3`, `tree-sitter`) on macOS arm64 + node v25 may lack prebuilds; codegraph advertises a WASM fallback. Any build failure is reported, not hidden.

Phase 2 — Initialize + index

`codegraph init`; tune `.codegraph/config.json` to index Lava domain code (`app/`, `core/`, `feature/`, `buildSrc/`, `proxy/`, `lava-api-go/`) and exclude `submodules/`, `build/`, `releases/`, `.git/`, `.gradle/`, `.lava-ci-evidence/`, `node_modules/`, and all §6.H secret paths. `codegraph index` → `codegraph status` confirms a real non-empty graph. `.codegraph/codegraph.db` added to `.gitignore`.

Phase 3 — Wire all 5 agents

`codegraph install` auto-configures Claude Code + OpenCode. Manual project-scoped MCP wiring for Qwen Code, Kimi CLI, Crush. Prefer committed project-scoped config so the wiring is reproducible per clone.

Phase 4 — Anti-bluff verification suite

Five layers, each with a §6.J falsifiability rehearsal (deliberate break → observed failure → revert), evidence to `.lava-ci-evidence/codegraph/`:

1. **Index reality** — `.codegraph/codegraph.db` exists, non-empty, contains real Lava symbols (`MainActivity`, `LavaApplication`, a known `lava-api-go` symbol).
2. **Query correctness** — `codegraph query <symbol>` returns the correct `file:line`.
3. **MCP server** — drive `codegraph serve --mcp` over stdio JSON-RPC: `initialize` → `tools/list` → real `codegraph_search` call → assert real data.
4. **Per-agent end-to-end (×5)** — run each agent non-interactively with a prompt that forces a codegraph MCP tool call; assert the agent's answer carries the correct codegraph-derived `file:line`. Falsifiability: with the MCP server disabled the test must fail. **Any agent that genuinely cannot be driven end-to-end is recorded as a documented gap — never faked.**
5. **Runner** — `scripts/verify-codegraph.sh` aggregates; exit non-zero on any failure.

Phase 5 — Documentation

`docs/CODEGRAPH.md` (what/why, install, init/scan, per-agent wiring, running the suite, troubleshooting). Update `CLAUDE.md`, `AGENTS.md`, `docs/CONTINUATION.md` (§6.S).

Phase 6 — §6.L 63rd + QWEN.md propagation

Advance §6.L counter 62→63 in `CLAUDE.md` with this invocation's narrative. Create `QWEN.md` (a thin pointer/inheritance file referencing `CLAUDE.md`) at repo root, in each of the 18 submodules, and in `lava-api-go`. Confirm the anti-bluff mandate is inherited in every scope (it already is, via the §6.AD pointer-block + `./constitution/` submodule); `QWEN.md` joins that inherited surface. No inherited rule is weakened.

Phase 7 — Commit, push, converge

Commit parent + each touched submodule; push to GitHub + GitLab; verify §6.C convergence; run `scripts/check-constitution.sh` and the pre-push gate.

7. Anti-bluff verification design detail

The verification suite's chief assertion (per Sixth Law clause 3) is always on **user-visible / real data**: a real symbol's real `file:line` as resolved from the real indexed Lava codebase, surfaced through the real MCP transport, observed in the real agent's real output. "The config file exists" and "the process started" are permitted secondary assertions only.

Falsifiability rehearsal per layer (recorded in `.lava-ci-evidence/codegraph/`):

- Layer 1: rename/remove the DB → reality test fails.
- Layer 2: query a symbol that does not exist → empty; query a real one → correct line.
- Layer 3: request a non-existent tool / wrong symbol → MCP error / empty result.
- Layer 4: point the agent at a config with the codegraph server removed → the agent cannot answer from the graph → test fails.

8. Risks / honest unknowns

- **Native module build** on node v25 arm64 — may need WASM fallback or may fail. Verified in Phase 1; reported truthfully.

- **Kimi CLI MCP config format** — exact file/location confirmed during Phase 3.
- **Agent non-determinism** — Phase 4 asserts the *correct answer appears* with bounded retries; the falsifiability rehearsal (server disabled → must fail) guards against a false green from the agent answering out of training knowledge rather than the graph.
- **Index scope** — submodules excluded initially; revisitable if the operator wants cross-submodule graph queries later.

9. Coverage exemptions

None anticipated. codegraph is third-party code — Lava does not test codegraph's internals; Lava tests that *codegraph, as integrated into Lava, works for Lava's agents*. That is the behavioral contract this spec's Phase 4 covers in full.